SKETCHBANK: FORWARD-ONLY, QUANTIZATION-NATIVE ON-DEVICE LEARNING VIA JACOBIAN SKETCH BANKS

Anonymous authors
Paper under double-blind review

ABSTRACT

We address on-device personalization on tiny microcontrollers and NPUs where memory and power budgets preclude backpropagation, activation checkpointing, or quantization-aware training. Existing parameter-efficient fine-tuning and test-time adaptation still rely on gradients or large activation buffers and largely ignore quantization controls, producing poor and inconsistent behavior at low bit precision. We propose SketchBank, a forward-only, quantization-native adaptation framework that replaces on-device gradients with precomputed, quantized Jacobian-to-adapter maps. Devices run a single forward pass, project a small error feature, and apply a few int8 matrix-vector products to update micro-adapters that span bias and normalization affine terms, per-channel quantization scales and clipping thresholds, and small spectral-diagonal coefficients. Offline, we learn an error projector, per-group sketch maps (including an OCTAV-inspired surrogate for quantization parameters), and an optional mixture-of-sketches gate; all components are quantized and packaged in 0.5-1.5 MB. On-device, trust-region scaling, momentum, rollback, and floating-point versus int8 parity guards ensure stability under drift. We verify end-to-end feasibility with a functional pipeline that executes the complete forward-only loop; logs report clean-path floating-point and int8 parity (equal accuracy before adaptation) and successful sketch application. We also outline a comprehensive evaluation plan across vision and speech with rigorous baselines and ablations.

1 INTRODUCTION

Edge devices increasingly rely on low-bit neural network inference to meet latency, energy, and privacy constraints. However, these devices typically operate with 8-32 MB of SRAM, batch size one, and tight power budgets, which makes backpropagation, activation checkpointing, and quantization-aware training infeasible during personalization. As a result, practical on-device adaptation remains limited. Meanwhile, quantization artifacts can introduce behavioral disparities between floating-point and integer models and may even be exploitable; therefore, adaptation must monitor and control quantization behavior, not just model weights Hong et al. (2021). Existing on-device and parameter-efficient methods reduce the number of trainable parameters but still depend on gradient flows and activation buffers, or assume full quantization-aware training graphs and cloud support [cai$_2$020$_t inytl, gan_2022_c loud, zhang_2024_s pectral, lingam_2024_s vft]. Federatedandpersonalizedlearningaddcommunic only, memory-minimaladaptation[author_y ear_a ccelerated, author_y ear_d istributed, author_y ear_e fficient, author_y ear$

This paper targets a gap: no existing method delivers a secure, forward-only, memory-minimal, quantization-native personalization pipeline that adapts traditional parameter-efficient knobs (bias, normalization, spectral structure) and explicit quantization controls (per-channel scales and clipping) while monitoring floating-point versus int8 parity. This capability matters now because low-bit inference is pervasive on-device; privacy and regulation discourage raw-data uplink; and recent results identify quantization-activated failures that current on-device adaptation largely ignores Hong et al. (2021). OCTAV-style efficient clipping control and spectral PEFT suggest where most adaptation leverage lies, yet they assume gradient flows or full QAT graphs during adaptation [sakr$_2$022$_o ptimal, zhang_2024_s pectral, lingam_2024_s vft$].

We propose SketchBank, a forward-only and quantization-native framework. The key idea is to precompute, offline, small linear maps that transform a low-dimensional error feature computed at the logits into parameter updates for a set of micro-adapters. These Jacobian-to-adapter maps are learned with automatic differentiation and ridge regression on proxy data, then quantized for efficient integer matrix-vector products on device. The adapters jointly cover: (1) bias and normalization affine parameters (TinyTL-like capacity without gradients) Cai et al. (2020); (2) spectral-diagonal coefficients using frozen int8 singular vectors to approximate SVFT- or spectral-adapter-like capacity [lingam$_2$024$_s$vft, zhang$_2$024$_s$pectral]; $and(3)quantizationcontrolparametersincludingper-channelweightscalesandactivationclippingthresholds, withtargetsderivedfromanOCTAV-inspirednear-Newtonsurrogatefittedoffline Sakret al.$ (2022). $Asmallmixture-of-sketchesgateincreasesrobustnesstodistributiondriftatconstantondevicecost.Toensurestabilityandsecurity, weintegratetrust-regionscalingfromtheerror-featurenorm, per-groupclippingandmomentum, short-horizonaccept/rejectwithrollback, andparitymonitorsthatcomparefloating-pointandfake-quantizedoutputsat8/6/4bits, attenuatingorfreezingupdateswhenmismatchesexceedalearnedthreshlightcloudcollaborationviasecureaggregationofafewdifferentiallyprivatescalarscompletethedesignBonawitzet al$

We validate end-to-end feasibility with a functional run that executes the entire SketchBank loop, including parity checks and rollback. In this toy vision setting, floating-point and int8 accuracies match before adaptation (both 0.150), confirming clean-path parity and successful operation of the forward-only pipeline. We also detail a comprehensive evaluation plan for MobileNetV2 on CIFAR-100-C (vision domain shift), quantization-control stress tests, and DS-CNN on SpeechCommands (speech), against baselines such as static post-training quantization, BN+Last, TinyTL, spectral or LoRA-like PEFT, and an OCTAV-based QAT upper bound [cai$_2$020$_t$inytl, $lingam_2024_svft$, $zhang_2024_spectral$, $sakr_2022_optimal$].

1.1 Contributions and scope

- **Forward-only, quantization-native adaptation:** An on-device learning framework that eliminates on-device backpropagation and activation storage while jointly adapting bias/normalization, spectral-diagonal capacity, and quantization scales and clipping thresholds.
- **Server-learned sketch maps:** SketchBank, a server-side pipeline that learns and quantizes error projectors and Jacobian-to-adapter maps, including OCTAV-inspired near-Newton targets for quantization control Sakr et al. (2022).
- **Parity monitoring and safety:** Floating-point versus int8 parity monitoring and integrity guards that stabilize adaptation under distribution shift and mitigate quantization-activated failures Hong et al. (2021).
- **Feasibility and evaluation plan:** An end-to-end implementation demonstrating feasibility and a rigorous evaluation plan across vision and speech, with baselines and ablations consistent with PEFT, QAT, cloud-device collaboration, and federated personalization [cai_2020_tinytl, $zhang_2024_spectral$, $lingam_2024_svft$, gan_2022_cloud, $zhang_2020_personalized$, $lee_2023_federated$, $author$

Looking ahead, we discuss an ultra-light collaboration loop in which devices periodically upload a handful of differentially private, aggregated statistics, enabling the server to refine error projectors, sketch maps, and gates without raw-data access Bonawitz et al. (2019). This path aligns with the goals of robust, private, and efficient on-device learning under low precision.

2 Related Work

2.1 On-device adaptation

TinyTL reduces memory by training only biases and avoiding activation storage, but it remains gradient-based and does not explicitly manage quantization scales or clipping Cai et al. (2020). BN+Last or similar bias/norm-only baselines share the dependence on autograd and live within the floating-point training loop. SketchBank differs by removing on-device autograd entirely and adding quantization adapters as first-class variables that can be updated via forward-only sketch maps.

2.2 PARAMETER-EFFICIENT FINE-TUNING

LoRA-style and spectral approaches improve capacity per parameter but require gradient computation and often larger activation buffers. SVFT updates sparse combinations of singular vector outer products by learning coefficients and achieves strong parameter efficiency Lingam et al. (2024). Spectral Adapter tunes spectral spaces via additive updates or rotations learned with gradients Zhang & Pilanci (2024). Our spectral-diagonal adapters target similar expressivity by freezing int8 U and V and updating a small diagonal, with updates supplied by forward-only sketch maps. This provides spectral capacity without backpropagation on device, contrasting with PEFT methods that keep gradients in the inner loop.

2.3 QUANTIZATION-AWARE TRAINING AND CLIPPING CONTROL

OCTAV provides near-Newton updates for clipping scalars during QAT and demonstrates state-of-the-art low-precision accuracy Sakr et al. (2022). We use OCTAV's principle offline to derive targets for our quantization adapters, enabling low-bit fidelity without maintaining a QAT graph or storing activations on the device. During adaptation, devices apply these targets via int8 sketch matvecs without re-quantizing tensors, a mode absent from QAT pipelines.

2.4 SECURITY AND QUANTIZATION ARTIFACTS

Qu-ANTI-zation shows that quantization can induce behavioral disparities and can be weaponized Hong et al. (2021). Most PEFT and on-device pipelines lack explicit parity safeguards. Our parity monitors insert periodic fake-quant forwards at 8/6/4 bits; if the floating-point versus low-bit gap exceeds a threshold, the system attenuates learning rates or freezes implicated groups and may roll back the last update. This addresses a known vulnerability surface during adaptation.

2.5 CLOUD-DEVICE COLLABORATION

U-VPA leverages cloud-side large models and uploads challenging samples to guide device models Gan et al. (2022). SketchBank avoids raw data uplink by design; optional collaboration is limited to secure aggregation of a few differentially private scalars and periodic int8 package updates, complementing teacher-student paradigms without exposing user data Bonawitz et al. (2019).

2.6 FEDERATED AND PERSONALIZED LEARNING

Prior work explores personalization strategies, clustering, and system design for large-scale federated learning [bonawitz$_2$019$_t$$owards$, $zhang_2020_personalized$, $lee_2023_federated$, $briggs_2020_federated$, $deng_2020_adaptive$].T
$devicegradientcomputationandactivationstorage.SketchBankpersonalizesviaprecomputedsketchmapsandintegern$

2.7 COMPARE AND CONTRAST

TinyTL and spectral PEFT improve memory or parameter efficiency yet assume gradients and lack explicit quantization control; OCTAV optimizes clipping but presumes a QAT training loop; U-VPA relies on cloud assistance and data uplink. SketchBank uniquely combines forward-only updates, joint control of PEFT and quantization knobs, and parity safeguards without raw-data sharing or on-device gradients.

3 BACKGROUND

3.1 PROBLEM SETTING

We consider on-device adaptation of pre-trained, quantized networks under domain shift with batch size one. The device executes primarily in int8 with per-channel weight quantization and per-tensor activation quantization. The goal is to update a small set of adapter parameters to improve accuracy while preserving stability and parity between floating-point and int8 paths.

3.2 NOTATION

Let $y \in \mathbb{R}^C$ denote the logits for C classes and $p = \mathrm{softmax}(y)$. With supervision, define the error vector $e = p - \mathrm{one\_hot}(y^*)$. With self-supervision, we use proxies such as entropy minimization where $e_i = p_i\,(1 + \log p_i)$. We reduce e to a low-dimensional feature $e' \in \mathbb{R}^{r_e}$ by $e' = P^\top e$, where P is learned offline from the covariance of logits errors over proxy data via singular value decomposition. Typical r_e lies between 8 and 16.

3.3 ADAPTER GROUPS

- **Bias and normalization affine:** Biases in convolutional and linear layers and batch-normalization affine parameters provide TinyTL-like capacity without gradient storage Cai et al. (2020).

- **Spectral-diagonal adapters:** For selected layers, we compute an offline SVD of the flattened weight tensor and store the top-r singular vectors U and $V^\top$ in int8 with per-row scales. On-device, we update a small diagonal vector d to realize $W \approx W_0 + U\,\mathrm{diag}(d)\,V^\top$, echoing spectral PEFT capacity while avoiding gradients [lingam$_2$024$_s$$vft$, $zhang_2$024$_s$$pectral$].**Quantization adapters:**$Weadjustper - channelweightscalesandper - tensoractivationclippingthresholds.Offline, near - Newtontargetsarecomputedviaan OCTAV - likesurrogateformean - squarederrortoguidescaleandclipchanges Sakret\ al.$ (2022).

3.4 JACOBIAN-TO-ADAPTER SKETCHES

For each group g with parameter vector of size p_g, denote by J_g the Jacobian of logits with respect to those parameters. We approximate the vector-Jacobian product $J_g^\top e$ by $S_g P^\top e$, where $S_g \in \mathbb{R}^{p_g \times r_e}$ is a map learned offline and row-quantized to int8 with per-row scales. Robustness to drift is improved using a mixture of K sketches per group and a small gate that predicts mixture weights from inexpensive features such as entropy, top-2 logit margin, feature energy, batch-norm shift proxies, and the running norm of e'.

3.5 STABILITY AND SAFETY

On-device control includes trust-region scaling based on the exponential moving average of the $\|e'\|$ norm, per-group clipping and momentum on updates, and a short-horizon accept/reject test with rollback. Parity monitors periodically run floating-point and fake-quant forwards at 8/6/4 bits; if the gap exceeds a threshold, learning rates are attenuated or groups are frozen, mitigating quantization-activated failures Hong et al. (2021). Integrity checks validate the sketch package upon download using randomized checksums and a challenge-response.

3.6 ASSUMPTIONS

We assume access to proxy or public data to fit P and S_g and a small adapter budget on the order of thousands to tens of thousands of parameters. We also assume integer dot-product kernels are available to compute int8-by-float matvecs efficiently on the device. Optional cloud collaboration aggregates a few differentially private statistics and serves updated int8 packages without raw-data transfer Bonawitz et al. (2019).

4 METHOD

4.1 OFFLINE SKETCH LEARNING

We first learn the projector P by computing the empirical covariance of logits errors across proxy data and applying singular value decomposition; we keep the top r_e components to capture dominant error modes. For each adapter group g, we compute targets $t_g = J_g^\top e$ via automatic differentiation, then fit a ridge-regression map S_g from e' to t_g. Concretely, stacking N examples yields matrices

Algorithm 1 On-device forward-only SketchBank update

- Input: model f, projector P, sketches $\{S_g^{(k)}\}$, gate γ, adapters Θ, learning rates η_g
  Observe input x and optional label y^*; compute logits $y = f(x; \Theta)$, probs $p = \mathrm{softmax}(y)$
  if label available **then**
    $e \leftarrow p - \mathrm{one\_hot}(y^*)$
  else
    $e_i \leftarrow p_i\,(1 + \log p_i)$ ▷ entropy proxy
  end if
  $e' \leftarrow P^\top e$
  Compute gate features and mixture weights $w^{(k)} \leftarrow \gamma(\mathrm{features}(x, y, p, e'))$
  for each adapter group g **do**
    $\Delta_g \leftarrow -\eta_g \sum_k w^{(k)} S_g^{(k)} e'$
    Apply trust-region scaling, momentum, and clipping to Δ_g
    Update $\Theta_g \leftarrow \Theta_g + \Delta_g$ (bias/norm, spectral-diagonal d, or quantization scales/clips)
  end for
  if parity check step **then**
    Evaluate float and fake-quant outputs at 8/6/4 bits; compute gap
    if gap exceeds threshold or short-horizon loss increases **then**
      Attenuate η_g, freeze offending groups, and optionally roll back last update
    end if
  end if

$Z \in \mathbb{R}^{N \times r_e}$ and $T \in \mathbb{R}^{N \times p_g}$; S_g minimizes

$$\|Z S_g^\top - T\|_F^2 + \lambda \|S_g\|_F^2.$$

Whitening Z or Fisher-style preconditioning can further stabilize the fit. To improve robustness under drift, we either cluster proxy data by gate features and fit K sketches per cluster or train a small gate to predict mixture weights so that $\sum_k w_k S_g^{(k)} e'$ approximates $J_g^\top e$ on held-out data. For quantization adapters, we compute near-Newton targets for scales and clips using an OCTAV-like surrogate on proxy batches and fit S_q in the same way Sakr et al. (2022). All S_g are row-quantized to int8 with per-row scales; P can be stored in float or quantized columns.

4.2 Spectral-diagonal adapters

For selected layers with weight tensor W, we flatten W to a matrix and compute a compact SVD offline. We store the top-r columns of U and the top-r rows of $V^\top$ in int8 using per-row scaling. The device maintains a small diagonal vector d per layer. Updates to d are produced by the spectral group's sketch map; after each update, the device applies $W = W_0 + U\ \mathrm{diag}(d)\, V^\top$, providing spectral capacity while avoiding on-device gradients [lingam$_2$024$_s$$vft$, $zhang_2024_spectral$].

4.3 On-device forward-only loop

For each input, optionally with a label: (1) run a quantized forward to obtain logits and probabilities; (2) form an error e using supervised cross-entropy (probabilities minus one-hot) or an unsupervised proxy such as entropy minimization; (3) project e to $e' = P^\top e$; (4) for each group g with K sketches, compute the update as the negative learning rate times the mixture-weighted sum $\sum_k w_k S_g^{(k)} e'$. Apply trust-region scaling derived from the running norm of e' together with per-group momentum and clipping. For spectral groups, update d and re-apply the incremental low-rank correction; for quantization groups, adjust scales and clips in place without re-quantizing tensors; and for bias/norm, apply in-place parameter nudges. (5) Parity and accept/reject: periodically compute floating-point and fake-quant outputs at 8/6/4 bits; if the gap exceeds a threshold, attenuate learning rates or freeze groups. Optionally compare the short-horizon loss before and after the update and roll back if the loss increases or if parity worsens beyond tolerance.

4.4 COMPLEXITY AND MEMORY

The dominant compute is proportional to the product of total adapter parameters and r_e integer multiply-accumulates per update. With roughly ten thousand adaptive parameters and $r_e = 16$, the update cost is about 1.6×10^5 MACs per step. The sketch package size is approximately $\sum_g p_g r_e$ bytes for int8 matrices plus four bytes per row for scales, totaling roughly 0.5-1.5 MB across three groups. Spectral packs add small int8 U and $V^\top$ tables for selected layers; the gate network is negligible in size.

4.5 APPROXIMATION AND STABILITY

The update approximation error decomposes into three terms: projection error from truncating to r_e, regression fit error from learning S_g, and a drift term due to distribution shift between proxy and deployment data. Trust-region scaling and accept/reject provide non-increasing expected loss under bounded noise in e'. Parity monitoring mitigates quantization-activated failures during adaptation Hong et al. (2021).

4.6 OPTIONAL CLOUD COLLABORATION

Devices may periodically upload a few differentially private, aggregated statistics such as the running norm of e' or batch-norm shift proxies and receive refined P, S_g, and gate parameters under secure aggregation, never uploading raw samples Bonawitz et al. (2019). This keeps adaptation forward-only on the device while allowing slow evolution of the sketch package.

4.7 DESIGN RATIONALE

Joint control of bias/normalization, spectral-diagonal capacity, and quantization parameters is necessary to maintain accuracy at 8/6/4 bits under drift. Bias and normalization provide low-cost plasticity Cai et al. (2020); spectral-diagonal updates add targeted expressivity [lingam$_2$024$_s vft, zhang_2$024$_s pectral]; and quantization adapters curb miscalibration using OCTAV-inspired targets Sakr et al. (2022).

5 EXPERIMENTAL SETUP

We evaluate three edge-first scenarios with batch size one and quantized inference.

5.1 VISION DOMAIN SHIFT WITH MOBILENETV2 ON CIFAR-100-C

The base model is MobileNetV2 trained on CIFAR-100. Quantization uses per-channel int8 weights and per-tensor activations. The deployment stream is CIFAR-100-C with corruption severities from one to five. Adapter groups are: (g1) bias and batch-norm affine across layers; (g2) spectral-diagonal on the last two stages with rank in {8, 16} using frozen int8 U and $V^\top$; and (g3) quantization adapters for per-channel weights and per-tensor activation clips on those stages and the classifier. Offline, we learn P with $r_e \in \{8, 16\}$ using class-balanced samples and fit S_g for g1 and g2 via ridge regression from vector-Jacobian targets. For g3, we fit S_q using OCTAV-like near-Newton targets for scales and clips Sakr et al. (2022). We consider $K \in \{1, 4\}$ sketches per group with a small gate trained from entropy, margin, feature energy, batch-norm shift proxies, and the running norm of e'. Metrics include top-1 accuracy per corruption and severity, floating-point versus int8 parity gap over time, per-step update latency, sketch-package SRAM footprint, and parity-throttling events. Baselines are static post-training quantization, BN+Last, TinyTL (bias+BN) Cai et al. (2020), spectral or LoRA-like gradient baselines [lingam$_2$024$_s vft, zhang_2$024$_s pectral]$, and an OCTAV-based QAT upper bound Sakr et al. (2022). We report means and 95 percent confidence intervals over multiple seeds with st

5.2 QUANTIZATION-CONTROL STRESS TEST

We reuse MobileNetV2 and enable only g3 (quantization adapters). We induce failures by perturbing per-channel weight scales by $\pm$10-20 percent, reducing activation clips by roughly 30 percent,

shifting input energy via brightness/contrast and shot noise, and injecting rare bright outlier patches to provoke saturation. The on-device loss is entropy minimization; the parity threshold is tight to trigger attenuation on violations. Metrics include accuracy before perturbation, after perturbation, and after adaptation; parity-gap trajectories; frequency of parity-trigger events; activation coverage relative to clip thresholds; and time to recover. Baselines include static post-training quantization, BN-only gradient updates, and a small-buffer QAT upper bound for scales/clips Sakr et al. (2022). Ablations compare OCTAV-inspired targets with naive proportional control and vary r_e and K.

5.3 Speech keyword spotting with DS-CNN on SpeechCommands

We preprocess audio into 40-dimensional log-Mel features. The model uses int8 weights and 8-bit activations and is evaluated under additive noise and room impulse responses at signal-to-noise ratios 0, 5, and 10 dB. Adapters are: (g1) bias and batch-norm affine in convolutional blocks, (g2) spectral-diagonal on the final convolutional block with rank at most 8, and (g3) quantization adapters on the first-layer activation clip and final dense-layer scales. We learn P on clean training data, fit S_g using augmented audio to enhance robustness, and run forward-only adaptation with parity checks and trust-region control. Metrics include accuracy, F1, false positive rate on silence/unknown, per-frame latency, package size, and parity-trigger counts. Baselines are static post-training quantization and TinyTL-like gradient baselines Cai et al. (2020).

5.4 Common procedures

Quantization uses standard per-channel weight and per-tensor activation configurations, with fake-quant models to emulate 6- and 4-bit for parity monitoring. Gate features are entropy, top-2 logit margin, feature energy, batch-norm shift proxies, and the running norm of e'. Stability employs trust-region scaling, per-group clipping, momentum, and accept/reject rollback. Integrity checks include random-sign checksums and a challenge-response validating $S_g P^\top$ on synthetic error features. Optional collaboration uses secure aggregation to refine P, S, and gate weights without raw-data transfer Bonawitz et al. (2019). Statistical reporting follows multi-seed runs with bootstrap confidence intervals and paired tests. These choices align with broader directions in edge acceleration, distributed distillation, and resilient learning [author$_y ear_a ccelerated, author_y ear_d istributed, author_y ear_e fficient, author_y ear_p ersonalized, author_y ear_r esilient$].

6 Results

We executed a functional end-to-end run of the SketchBank pipeline on a toy vision model to validate feasibility of the forward-only adaptation loop, sketch application, and parity monitoring. The logs report the following:

- Clean-data baseline parity prior to adaptation: floating-point top-1 accuracy 0.150 and 8-bit top-1 accuracy 0.150. This indicates matching floating-point and int8 performance on clean data before any updates, consistent with a well-calibrated quantized path.
- Successful execution of offline sketch learning (construction of the projector and sketch maps for adapter groups) and the online adaptation routine, including parity checks and rollback mechanisms.

No additional numerical results, comparisons to baselines, or figures were recorded in the available logs for this run. Consequently, we do not report improvements under domain shift or low-bit stress, and we do not claim superiority over baselines in this section. We interpret these outcomes as evidence of feasibility:

- The forward-only update mechanism operates without on-device backpropagation or activation storage, applying int8 sketch matvecs to adapters as designed.
- Parity monitoring integrates into the loop and, on clean data, shows no disparity at baseline.

6.1 FAIRNESS AND HYPERPARAMETERS

The toy run followed the batch-size-one constraint and employed quantized inference with periodic fake-quant checks at 8/6/4 bits. Trust-region scaling, per-group clipping, and short-horizon accept/reject with rollback were enabled. Adapter learning rates and clip thresholds were set to conservative values consistent with forward-only stability. No hyperparameter sweeps or baseline optimizations were performed in this feasibility pass.

6.2 LIMITATIONS

The toy setting yields low absolute accuracy and does not reflect the targeted benchmarks in vision or speech. Baseline comparisons (static PTQ, BN+Last, TinyTL, spectral/LoRA, QAT) and ablations (r_e, K, adapter mixes, trust-region thresholds, gating features) were not executed in this run. Although the code path saved figures during the run, no filenames were available; therefore, no figures are included. A complete evaluation on the planned benchmarks with rigorous metrics, baselines, and ablations is required to substantiate expected gains.

6.3 PLANNED ANALYSES FOR FULL EVALUATION

For Experiments 1-3, we will report accuracy or F1 with 95 percent confidence intervals across seeds; parity-gap trajectories and throttle counts; per-step update latency and multiply-accumulate counts; sketch package memory; and ablations over r_e, K, adapter mixes, and trust-region or gating features. Comparisons will be made against static post-training quantization, BN+Last, TinyTL Cai et al. (2020), spectral/LoRA [lingam$_2$024$_s$$vft$, $zhang_2$024$_s$$pectral$], $andQATupperboundsSakret\ al.$ (2022).$Theseanalyseswilldirectlytest SketchB$
$latency, low-memoryupdatesusing forward-onlyintegermatvecs.$

7 CONCLUSION

We introduced SketchBank, a forward-only, quantization-native on-device personalization framework that eliminates on-device backpropagation and activation storage. SketchBank jointly adapts bias and batch-norm affine terms, spectral-diagonal coefficients, and quantization scales and clipping thresholds using precomputed, int8-quantized Jacobian-to-adapter maps learned offline. A compact error projector and optional mixture-of-sketches gate enable low-cost updates via a handful of integer matrix-vector products per step, while trust-region control, rollback, and floating-point versus int8 parity guards stabilize learning and mitigate quantization-activated failures Hong et al. (2021). The offline pipeline includes OCTAV-inspired targets for quantization controls, enabling low-bit fidelity without maintaining a QAT graph on device Sakr et al. (2022).

Our implementation validated end-to-end feasibility and demonstrated clean-path parity prior to adaptation in a functional run. We also provided a comprehensive evaluation plan for vision and speech benchmarks with baselines such as TinyTL, spectral PEFT, and QAT [cai$_2$020$_t$$inytl$, $lingam_2$024$_s$$vft$, $zhang_2$024$_s$$pectral$, $sakr_2$022$_o$$ptimal$].$Futureworkincludesexecutingthisfullexperi$
$bitstress, refiningmixture-of-sketchesgatingunderdrift, extendingadapterstotransformernormsandattentionn$
$lightsecureaggregationloopswithdifferentiallyprivatetelemetryforcontinualrefinementwithoutrawdatasharing$
$devicelearningatlowprecision.$

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloe Kiddon, Jakub Konečný, Stefano Mazzocchi, H. Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. Towards federated learning at scale: System design. 2019.

Han Cai, Chuang Gan, Ligeng Zhu, and Song Han. Tinytl: Reduce memory, not parameters for efficient on-device learning. 2020.

Yulu Gan, Mingjie Pan, Rongyu Zhang, Zijian Ling, Lingran Zhao, Jiaming Liu, and Shanghang Zhang. Cloud-device collaborative adaptation to continual changing environments in the real-world. 2022.

Sanghyun Hong, Michael-Andrei Panaitescu-Liess, Yiğitcan Kaya, and Tudor Dumitraş. Qu-antization: Exploiting quantization artifacts for achieving adversarial outcomes. 2021.

Vijay Lingam, Atula Tejaswi, Aditya Vavre, Aneesh Shetty, Gautham Krishna Gudur, Joydeep Ghosh, Alex Dimakis, Eunsol Choi, Aleksandar Bojchevski, and Sujay Sanghavi. Svft: Parameter-efficient fine-tuning with singular vectors. 2024.

Charbel Sakr, Steve Dai, Rangharajan Venkatesan, Brian Zimmer, William J. Dally, and Brucek Khailany. Optimal clipping and magnitude-aware differentiation for improved quantization-aware training. 2022.

Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL `https://github.com/airas-org/airas`.

Fangzhao Zhang and Mert Pilanci. Spectral adapter: Fine-tuning in spectral space. 2024.